PPGCC
PROGRAMA DE PÓS-GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO

INF
INSTITUTO DE INFORMÁTICA

UFG
UNIVERSIDADE FEDERAL DE GOIÁS

APOSTILA TÉCNICA

Desenvolvimento de DSL com Langium e Integração com Grafos no Neo4j

Um guia prático para configurar ambientes, modelar linguagens de domínio específico e preparar arquiteturas estruturadas para Modelos de Linguagem (LLMs).

Mestrando: Fernando Souza Furtado Carrilho
Orientador: Prof. Dr. Fábio Moreira Costa

Universidade Federal de Goiás
Instituto de Informática (INF) | PPGCC

Goiânia - GO · 2026

FERNANDO SOUZA FURTADO CARRILHO

Desenvolvimento de DSL com Langium e Integração com Grafos no Neo4j

Apostila técnica e material de apoio metodológico desenvolvidos no contexto da pesquisa de Mestrado apresentada ao Programa de Pós-Graduação em Ciência da Computação (PPGCC) do Instituto de Informática da Universidade Federal de Goiás (UFG).

Este documento visa guiar a configuração de ambientes de Engenharia Dirigida a Modelos e sua integração em arquiteturas com Modelos de Linguagem (LLMs).

Orientador: Prof. Dr. Fábio Moreira Costa

Goiânia - GO
2026

Conteúdo

1 Visão Geral **4**

2 Setup do Ambiente **4**
2.1 Instalações Necessárias 4
2.2 Extensões do VS Code 4

3 Criação do Projeto **5**
3.1 Inicialização e Dependências Básicas 5
3.2 Dependências do Language Server e Tipagem 5
3.3 Configuração TypeScript 5

4 Estrutura do Projeto **6**

5 Criando sua DSL **6**
5.1 Definição da Gramática 6
5.2 Exemplo de Uso 7
5.3 Teste do Langium 7

6 Langium - Pipeline de Parsing **7**

7 Gerando a Linguagem **9**

8 Trabalhando com AST **9**

9 Integração com Neo4j **10**
9.1 Instalação do Driver 10
9.2 Conexão e Inserção de Dados 10

10 Integração: Backend e Evolução **11**
10.1 Plano de Testes 11
10.2 Evolução da DSL 12

11 Arquivos dos diretórios **13**
11.1 DSL - Module 13
11.2 DSL - Langium 14
11.3 DSL - Language Server 15
11.4 DSL - Extension 16
11.5 DSL - Test 17
11.6 DSL - Package.JSON 18
11.7 DSL - TSConfig.JSON 19

Lista de Figuras

1 Pipeline de parse do Langium com espaçamento e setas corrigidas. . . . 8

1. Visão Geral

Objetivo de Aprendizagem

Capacitar o leitor a configurar o ambiente de desenvolvimento, criar uma DSL (*Domain-Specific Language*), integrar com bancos de dados orientados a grafos (Neo4j) e preparar a base arquitetural para uso futuro com LLMs.

O fluxo do sistema segue uma arquitetura moderna e sequencial, transformando texto em estruturas semânticas ricas:

Fluxo da Arquitetura

```
DSL (Texto) -> AST (Arvore de Sintaxe) -> Grafo (Neo4j) ->
    LLM (Orquestracao)
```

2. Setup do Ambiente

Para garantir que todos os componentes funcionem em harmonia, o ambiente de desenvolvimento deve ser configurado com ferramentas modernas e atualizadas.

2.1 Instalações Necessárias

- **VS Code:** Editor de código recomendado.
- **Node.js (LTS):** Motor de execução para o ecossistema Langium.
- **Python 3.10+:** Para integrações futuras de inteligência artificial.

2.2 Extensões do VS Code

Recomenda-se a instalação das seguintes extensões para suporte completo a tipagem e formatação:

- Python
- ESLint
- Prettier
- Node.js Extension Pack

3. Criação do Projeto

3.1 Inicialização e Dependências Básicas

Inicie criando o diretório do projeto e configurando as dependências centrais do Node e do Langium. O Langium será o núcleo para a construção da nossa gramática e do *parser*.

Terminal

```
mkdir dsl-project
cd dsl-project
npm init -y
npm install langium
npm install -D typescript ts-node
```

3.2 Dependências do Language Server e Tipagem

Para que a sua DSL possua integração total com o VS Code (autocompletar, validação de erros em tempo real e realce de sintaxe), precisamos instalar os pacotes oficiais do *Language Server Protocol* (LSP) e as tipagens do Node.js.

Terminal

```
npm install vscode-languageclient vscode-languageserver
npm install vscode-languageserver@9
npm install -D @types/node
```

3.3 Configuração TypeScript

Com as dependências instaladas, inicializamos a configuração do compilador TypeScript, que guiará a transpilação do nosso código.

Terminal

```
npx tsc --init
```

4. Estrutura do Projeto

Mantenha seu código organizado. Uma estrutura limpa facilita a manutenção da linguagem de domínio e a integração com o VS Code.

Estrutura Atualizada do Projeto

```
dsl-project/
|-- assets/
|-- node_modules/
|-- src/
      |-- cli/
      |-- generated/
      |-- language/
            |-- dsl-module.ts
            +-- dsl.langium
      |-- language-server/
            +-- main.ts
      |-- syntaxes/
      +-- extension.ts
|-- .gitignore
|-- langium-config.json
|-- package-lock.json
|-- package.json
+-- tsconfig.json
```

5. Criando sua DSL

5.1 Definição da Gramática

A gramática é o coração da sua DSL. Aqui definimos como a linguagem deve ser escrita e compreendida pelo *parser*.

my-dsl.langium

```
grammar MyDsl

entry Model:
    elements+=Reservation*;

Reservation:
    'reserve' resource=ID 'at' time=STRING;
```

5.2 Exemplo de Uso

Com a linguagem definida, o usuário final poderá escrever instruções da seguinte forma:

script.mydsl

```
reserve SalaA at "2026-03-22 14:00"
reserve SalaB at "2026-03-22 16:00"
```

5.3 Teste do Langium

Para rodar a execução do exemplo de Uso (descrito no subtópico 5.2), e ver sua saída (do teste.ts, do subtópico 11.5), basta rodar o comando a seguir:

No Terminal

```
npm run langium:generate
npm run build
npx tsc && node out/test/test.js
```

6. Langium - Pipeline de Parsing

O processo de transformação do texto DSL em uma saída estruturada segue as seguintes etapas fundamentais, na Figura 1:

FLUXO DE OPERAÇÃO DO LANGIUM

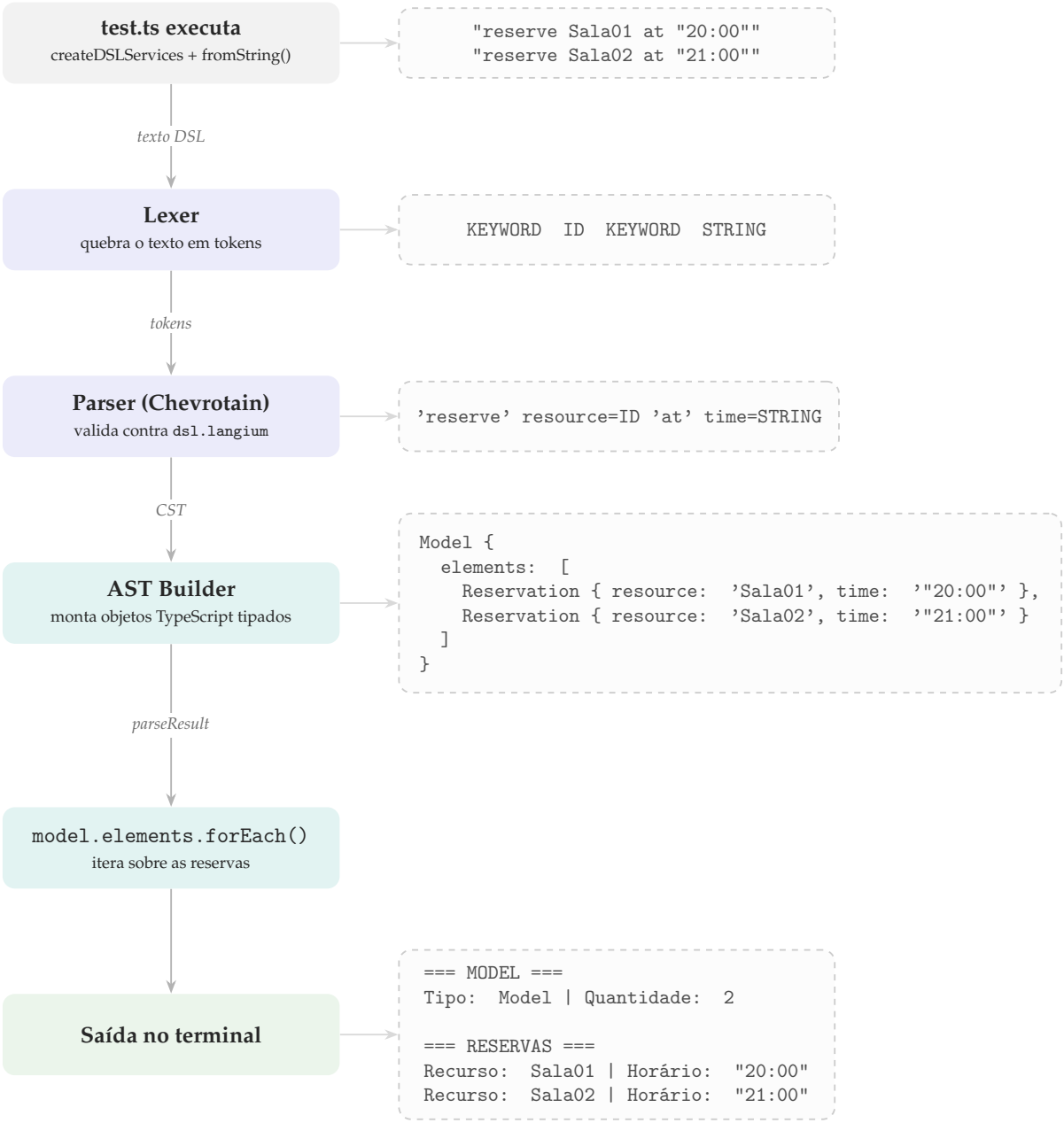

Figura 1: Pipeline de parse do Langium com espaçamento e setas corrigidas.

1. **Entrada de Dados:** O arquivo `test.ts` invoca o método `fromString()` passando o conteúdo textual bruto:

   ```
   "reserve Sala01 at "20:00"\n reserve Sala02 at "21:00""
   ```

2. **Análise Léxica (Lexer):** O *Lexer* percorre o texto caractere por caractere, agrupando-os em unidades lógicas chamadas *tokens*. Palavras-chave como `'reserve'` e `'at'` são identificadas como `KEYWORD`, identificadores como `'Sala01'` como `ID`, e valores entre aspas como `STRING`.

3. **Análise Sintática (Parser):** Utilizando a biblioteca *Chevrotain*, o *Parser* valida se a sequência de *tokens* obedece às regras definidas no arquivo `dsl.langium`.

   - Caso a sequência seja inválida, são gerados `parserErrors`.
   - Caso seja válida, o parser identifica cada estrutura de `Reservation` e extrai seus respectivos valores.

4. **Construção da AST (AST Builder):** O resultado do parser é convertido em uma *Abstract Syntax Tree* (AST). São instanciados objetos TypeScript tipados, onde o objeto raiz `Model` contém um array `elements[]` composto por instâncias de `Reservation`, cada uma possuindo os atributos `.resource` e `.time`.

5. **Execução e Saída:** Por fim, o fluxo percorre a árvore através de um `model.elements.forEach()`. Para cada elemento, os dados são processados e enviados para o console (`console.log`), gerando a visualização final estruturada no terminal.

7. Gerando a Linguagem

Utilize o Langium CLI para ler o arquivo de gramática e compilar a Árvore de Sintaxe Abstrata (AST) e os serviços TypeScript necessários.

Terminal

```
npm install langium-cli -D
npx langium generate
```

8. Trabalhando com AST

Após a interpretação do texto, a AST permite navegar programaticamente pelos elementos gerados pela DSL.

main.ts

```
model.elements.forEach(e => {
    console.log(`Recurso: ${e.resource} | Horario: ${e.time
        }`);
});
```

9. Integração com Neo4j

Grafos de conhecimento fornecem o contexto espacial e relacional perfeito para as regras de negócio validadas pela sua DSL.

9.1 Instalação do Driver

Terminal

```
npm install neo4j-driver
```

9.2 Conexão e Inserção de Dados

O código abaixo demonstra como instanciar a conexão e persistir os nós originados da sua AST.

database.ts

```
import neo4j from 'neo4j-driver';

// Inicializando a conexao
const driver = neo4j.driver(
  "bolt://localhost:7687",
  neo4j.auth.basic("neo4j", "senha_super_secreta")
);

const session = driver.session();

// Inserindo dados dinamicos da AST
await session.run(
  "CREATE (r:Reservation {resource: $resource, time: $time})
     ",
  { resource: e.resource, time: e.time }
);
```

10. Integração: Backend e Evolução

Fluxo Multi-Camadas

A arquitetura sugere uma esteira de dados robusta, separando as responsabilidades de parsing, armazenamento e inferência inteligente.

Pipeline de Integracao

```
DSL -> Node (Langium) -> JSON -> API Python -> Neo4j -> LLM
```

10.1 Plano de Testes

Para garantir a integridade do sistema, implemente:

- Testes de sintaxe para rejeitar comandos de DSL inválidos.
- Validação de inserção e integridade dos nós no grafo Neo4j.

- Verificação de cardinalidade e relações entre entidades.

10.2 Evolução da DSL

Conforme o domínio do problema cresce, a DSL pode se adaptar com novas palavras-chave e ramificações condicionais.

Evolucao - my-dsl.langium

```
Reservation:
  'reserve' resource=ID
  'at' time=STRING
  ('ifAvailable')?; // Tornando a restricao opcional
```

Conclusão do Módulo

Com este processo, você construiu os alicerces de uma DSL integrada a grafos. Esta fundação sólida, blindada por uma gramática formal e enriquecida por propriedades relacionais, está agora perfeitamente preparada para atuar como *grounding* em arquiteturas avançadas com LLMs.

11. Arquivos dos diretórios

11.1 DSL - Module

src/language/dsl-module.ts

```
import {
    createDefaultCoreModule,
    createDefaultSharedCoreModule,
    inject
} from 'langium';

import {
    MyDslGeneratedModule,
    dslProjectGeneratedSharedModule
} from '../generated/module.js';

export function createDSLServices(context: any) {
    const shared = inject(
        createDefaultSharedCoreModule(context),
        dslProjectGeneratedSharedModule
    );

    const DSL = inject(
        createDefaultCoreModule({ shared }),
        MyDslGeneratedModule
    );

    return { shared, DSL };
}
```

11.2 DSL - Langium

src/language/dsl.langium

```
grammar MyDsl

hidden terminal WS: /\s+/;

entry Model:
    elements+=Reservation*;

Reservation:
    'reserve' resource=ID 'at' time=STRING;

terminal ID: /[a-zA-Z_][a-zA-Z0-9_]*/;
terminal STRING: /"[^"]*"|'[^']*'/;
```

11.3 DSL - Language Server

src/language-server/main.ts

```
import {
    createDefaultCoreModule,
    createDefaultSharedCoreModule,
    inject
} from 'langium';

import {
    MyDslGeneratedModule,
    dslProjectGeneratedSharedModule
} from '../generated/module.js';

export function createDSLServices(context: any) {

    const shared = inject(
        createDefaultSharedCoreModule(context),
        dslProjectGeneratedSharedModule
    );

    const DSL = inject(
        createDefaultCoreModule({ shared }),
        MyDslGeneratedModule
    );

    return { shared, DSL };
}
```

11.4 DSL - Extension

src/extension.ts

```
import {
    createDefaultCoreModule,
    createDefaultSharedCoreModule,
    inject
} from 'langium';

import {
    MyDslGeneratedModule,
    dslProjectGeneratedSharedModule
} from '../generated/module.js';

export function createDSLServices(context: any) {

    const shared = inject(
        createDefaultSharedCoreModule(context),
        dslProjectGeneratedSharedModule
    );

    const DSL = inject(
        createDefaultCoreModule({ shared }),
        MyDslGeneratedModule
    );

    return { shared, DSL };
}
```

11.5 DSL - Test

src/test/test.ts

```
import { createDSLServices } from '../language/dsl-module.js';
import { EmptyFileSystem } from 'langium';
import { URI } from 'vscode-uri';
import { Model } from '../generated/ast.js';

async function run() {
    const { shared, DSL } = createDSLServices(EmptyFileSystem);

    const document = shared.workspace.LangiumDocumentFactory.fromString(
        `reserve Sala01 at "20:00"
         reserve Sala02 at "21:00"`,
        URI.parse('file:///tmp/test.dsl')
    );

    const result = document.parseResult;
    const model = document.parseResult.value as Model;

    console.log('=== PARSE RESULT ===');
    console.log(result);
    console.log('=== MODEL ===');
    console.log('Tipo:', model.\$type);
    console.log('Quantidade de reservas:', model.elements.length);

    console.log('\n=== RESERVAS ===');
    for (const el of model.elements) {
        console.log(`- Recurso: \${el.resource}`);
        console.log(`  Hor rio: \${el.time}`);
    }
}

run();
```

11.6 DSL - Package.JSON

src/package.json

```
{
  "name": "dsl-project",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "langium:generate": "langium generate",
    "build": "npm run langium:generate",
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "module",
  "dependencies": {
    "langium": "^4.2.1",
    "vscode-languageclient": "^9.0.1",
    "vscode-languageserver": "^9.0.1"
  },
  "devDependencies": {
    "@types/node": "^25.5.0",
    "@types/vscode": "^1.110.0",
    "langium-cli": "^4.2.0",
    "ts-node": "^10.9.2",
    "typescript": "^5.9.3"
  }
}
```

11.7 DSL - TSConfig.JSON

src/tsconfig.json

```
{
  "compilerOptions": {
    "module": "ESNext",
    "moduleResolution": "Bundler",
    "target": "ES2020",
    "rootDir": "src",
    "outDir": "out",
    "types": ["node", "vscode"],
    "strict": true,
    "skipLibCheck": true,
    "esModuleInterop": true
  }
}
```